

Planning and Assessment

Series: M2S | Notebook: 3 of 8 | Created: January 2026

Table of Contents

- 1. [Introduction](#)
- 2. [Discovery: Inventory Your Environment](#)
- 3. [Strategy: Define Your Approach](#)
- 4. [Design: Create Your Plan](#)
- 5. [Risk Assessment](#)
- 6. [Next Steps](#)

Prerequisites

Before starting this notebook, you should have:

Requirement	Description
Completed M2S-01 & M2S-02	Framework understanding
API Token	With <code>entities.read</code> , <code>settings.read</code> , <code>metrics.read</code> scopes
Admin Access	To Dynatrace Managed environment
Spreadsheet Tool	For inventory documentation

Learning Objectives

By the end of this notebook, you will:

- Complete a thorough discovery of your Managed environment
 - Document all configurations requiring migration
 - Define success criteria for your migration
 - Create a risk assessment matrix
 - Have a preliminary migration plan
-

1. Introduction

The Plan phase is critical—most migration problems stem from incomplete discovery or unclear strategy. This notebook guides you through systematic planning.

Why Planning Matters

Planning Gap	Consequence
Missing host inventory	Incomplete migration
Unknown integrations	Broken workflows
Unclear timeline	Resource conflicts
No success criteria	Uncertain completion

2. Discovery: Inventory Your Environment

2.1 Entity Inventory

Run these queries to build your entity inventory:

```
// Complete host inventory with details
fetch dt.entity.host
| fieldsAdd
    hostName = entity.name,
    osType,
    cloudType,
    agentVersion = entity.detected.properties[`oneagent.version`]
| fields hostName, osType, cloudType, agentVersion
| sort hostName asc
```

```
// Service inventory with technology
fetch dt.entity.service
| fieldsAdd
    serviceName = entity.name,
    serviceType,
    technologyType = entity.detected.properties[`technology`]
| fields serviceName, serviceType, technologyType
| sort serviceName asc
```

```
// Application inventory
fetch dt.entity.application
| fieldsAdd appName = entity.name, applicationType
| fields appName, applicationType
| sort appName asc
```

```
// Synthetic monitor inventory
fetch dt.entity.synthetic_monitor
| fieldsAdd monitorName = entity.name
| fields monitorName, id
| sort monitorName asc
```

2.2 Summary Statistics

Create a high-level summary for planning:

```
// Entity count summary
fetch dt.entity.host | summarize hosts = count()
| append [fetch dt.entity.service | summarize services = count()]
| append [fetch dt.entity.application | summarize applications = count()]
| append [fetch dt.entity.process_group | summarize processGroups = count()]
| append [fetch dt.entity.synthetic_monitor | summarize syntheticMonitors = count()]
```

```
// Hosts by OS type
fetch dt.entity.host
| summarize count(), by:{osType}
| sort `count()` desc
```

```
// Hosts by cloud provider
fetch dt.entity.host
| summarize count(), by:{cloudType}
| sort `count()` desc
```

2.3 ActiveGate Inventory

```
// ActiveGate inventory
fetch dt.entity.active_gate
| fieldsAdd gateName = entity.name
| fields gateName, id
```

2.4 Configuration Inventory

Document these configuration categories:

Category	Items to Document	Migration Method
Management Zones	Zone names, rules, permissions	Settings API export
Auto-tagging Rules	Tag definitions, conditions	Settings API export
Alerting Profiles	Profiles, severities, filters	Manual or API
Problem Notifications	Integrations, webhooks	Recreate with new URLs
Maintenance Windows	Scheduled windows	Manual recreation
Custom Dashboards	Dashboard JSON	Export/Import
SLOs	Definitions, targets	Settings API
Calculated Services	Service detection rules	Manual review
Request Attributes	Capture rules	Settings API

2.5 Integration Inventory

Integration Type	Details to Capture
ITSM	ServiceNow, Jira URLs and credentials
Notification	Email, Slack, Teams, PagerDuty configs
CI/CD	Jenkins, Azure DevOps, GitLab hooks
Custom API	Any scripts calling Dynatrace APIs
SSO/SAML	Identity provider configuration

Discovery Checklist

Entities

☐ Hosts (count by OS)

☐ Services (by type)

☐ Applications

☐ Process Groups

☐ Synthetic Monitors

☐ ActiveGates

☐ OneAgent Versions

Configurations

☐ Management Zones

☐ Auto-tagging Rules

☐ Alerting Profiles

☐ Dashboards

☐ SLOs

☐ Request Attributes

☐ Maintenance Windows

Integrations

☐ ITSM (ServiceNow)

☐ Notifications

☐ CI/CD Pipelines

☐ Custom Webhooks

☐ SSO/SAML Config

☐ API Consumers

☐ User Permissions

3. Strategy: Define Your Approach

3.1 Migration Approach Decision

Based on your discovery, choose your approach:

Your Environment	Recommended Approach
< 500 hosts, simple integrations	Big Bang
500-2000 hosts, moderate complexity	Phased by environment
> 2000 hosts, complex integrations	Phased by region/application
Strict compliance requirements	Phased with extended validation

3.2 Licensing Considerations

Important: Dual licensing and contract alignment are critical considerations for migration planning.

Consideration	Description
Dual-run period	Both Managed and SaaS may run simultaneously during migration
License overlap	Coordinate with Dynatrace for temporary overlap licensing
Contract timing	Align SaaS contract start with migration timeline
Host counting	Hosts appear in both environments during dual-run

Work with your Dynatrace account team to:

- Clarify licensing for the migration period
- Align contract terms with migration timeline
- Understand any temporary cost implications
- Plan the decommissioning of Managed licenses

3.3 Historic Data Considerations

 **Critical:** Historic monitoring data cannot be migrated from Managed to SaaS.

Data Type	Migration Status
Metrics history	✗ Cannot migrate
Log history	✗ Cannot migrate
Trace history	✗ Cannot migrate
Problem history	✗ Cannot migrate
User session data	✗ Cannot migrate
Configurations	✓ Can migrate via API

Planning implications:

- Start SaaS data collection before decommissioning Managed
- Consider dual-run period for baseline comparison
- Export any critical reports before shutdown
- Maintain Managed access for historical reference (if needed)

3.4 Success Criteria Definition

Define measurable success criteria:

Criterion	Target	Measurement
Host coverage	100% of current hosts reporting	DQL entity count
Service discovery	All services detected	DQL service count
Data continuity	No gaps > 15 minutes	Timeseries query
Alert functionality	Test alerts received	Notification test
Dashboard accuracy	All tiles showing data	Visual inspection
Integration health	All webhooks functioning	Integration tests

3.5 Timeline Planning

Phase	Duration	Key Milestones
Plan	Week 1-2	Discovery complete, strategy approved
Prepare	Week 3	SaaS tenant ready, network configured
Execute	Week 4	Agents migrated, configs applied
Validate	Week 5	All success criteria met
Optimize	Week 6+	Tuning and feature adoption

Best Practice: Minimize the dual-run period to reduce complexity and cost, but allow enough time for proper validation.

3.6 Stakeholder Matrix

Stakeholder	Role	Communication Frequency
Executive Sponsor	Approval, escalation	Weekly status
Platform Team	Technical execution	Daily standups
Application Teams	Validation, feedback	Phase notifications
Security Team	Compliance sign-off	Design review, final approval
Dynatrace Account Team	Support, guidance	As needed

4. Design: Create Your Plan

4.1 Migration Order of Operations

Follow this proven sequence for successful migration:

Step	Phase	Activity	Details
1	Plan	Assess	Inventory current environment - entities, configs, integrations
2	Prepare	Provision	SaaS tenant and access (SSO setup, IAM configuration)
3	Prepare	Install	New ActiveGates in parallel with old ActiveGates
4	Execute	Migrate	Configuration and integrations (via API/tooling)
5	Execute	Rebuild	Dashboards, management zones, alerts (non-portable items)
6	Execute	Redirect	OneAgents to SaaS (repoint or reinstall)
7	Execute	Reconnect	Integrations and extensions (validate connectivity)
8	Execute	Migrate	Any remaining configuration and integrations
9	Validate	Validate	Data flow, performance, coverage
10	Cutover	Cutover	Full switch to SaaS, notify stakeholders
11	Run	Decommission	Managed environment shutdown

Note: Steps 4-8 may overlap and iterate. The sequence ensures dependencies are satisfied (e.g., configurations in place before agents redirect).

4.2 Network Design

Component	Current (Managed)	Target (SaaS)
Cluster endpoint	Internal cluster URL	<code>{tenant}.live.dynatrace.com</code>
OneAgent routing	Cluster ActiveGate	Environment ActiveGate or direct
Firewall rules	Internal only	Outbound 443 to SaaS

4.3 ActiveGate Strategy

Scenario	Recommendation
Direct internet access	OneAgent direct to SaaS
Proxy required	Configure proxy on OneAgent
No internet access	Environment ActiveGate for routing
Multi-region	Regional Environment ActiveGates

4.4 Configuration Migration Sequence

Migrate configurations BEFORE redirecting OneAgents:

Order	Configuration Type	Why This Order
1	Deep monitoring settings	Affects how services are detected
2	Custom process monitoring rules	Affects process group detection
3	Container injection rules	Affects Kubernetes monitoring
4	Management zones	Needed for proper organization
5	Auto-tagging rules	Needed for filtering/organizing
6	Service detection rules	Affects service discovery
7	Request attributes	Needed for request data capture

Migrate AFTER OneAgents redirect (entities must exist):

Order	Configuration Type	Why This Order
8	Entity-level settings	Requires entity to exist in SaaS
9	Dashboards	Need data to validate
10	Alerting profiles	Need entities for filtering
11	Problem notifications	Need integrations validated

4.5 Rollback Plan

Scenario	Rollback Action	Recovery Time
OneAgent issues	Reconfigure to Managed endpoint	Minutes
Data quality issues	Pause migration, investigate	Hours
Complete failure	Full rollback to Managed	Hours

5. Risk Assessment

5.1 Risk Matrix

Risk	Likelihood	Impact	Mitigation
Network connectivity issues	Medium	High	Pre-test connectivity, have rollback
Configuration mismatch	Medium	Medium	Thorough discovery, validation queries
Integration failures	Medium	High	Test integrations early, document endpoints
Data gaps during migration	Low	Medium	Parallel operation period
User access issues	Low	Medium	SSO configuration testing
Performance degradation	Low	High	Monitor during migration, capacity planning

5.2 Go/No-Go Checklist

Before proceeding to Execute phase:

Checkpoint	Status
Discovery documented	[]
Strategy approved	[]
Network connectivity verified	[]
SaaS tenant provisioned	[]
API tokens created	[]
Rollback plan documented	[]
Stakeholders notified	[]
Maintenance window scheduled	[]

6. Next Steps

Immediate Actions

1. **Complete discovery queries** - Run all queries in this notebook
2. **Document configurations** - Create configuration inventory spreadsheet
3. **Define success criteria** - Get stakeholder agreement
4. **Create timeline** - Schedule phases and milestones
5. **Identify risks** - Complete risk assessment

Continue the Series

Next Notebook	Focus
M2S-04: Architecture & Design	Network topology and ActiveGate placement

Planning Resources

- [Dynatrace API Documentation](#)
- [Settings API Reference](#)

- [ActiveGate Sizing](#)
-

Summary

In this notebook, you learned:

- How to conduct thorough environment discovery
- DQL queries for building entity inventory
- Configuration categories requiring migration
- How to define success criteria and timeline
- Risk assessment methodology

Key Takeaway: Thorough planning prevents migration problems. Invest time in discovery—every unknown configuration is a potential issue during execution.

Continue to M2S-04: Architecture & Design for detailed technical planning.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.